

Reading a connectome out of a graph network: the template readout and the gauge it has to undo

1 What is observed, and what is free

The generator is a conductance-based network of N neurons. For neuron i ,

$$\tau_i \frac{dv_i}{dt} = -(v_i - V_i^{\text{rest}}) + \text{msg}_i + I_i, \quad \text{msg}_i = \sum_{j \in \mathcal{N}(i)} W_{ij} \text{act}(v_j) (E_{ij} - v_i), \quad (1)$$

with $\text{act} = \text{relu}$ on this data, $W_{ij} \geq 0$ a synaptic conductance in units of the leak conductance, and E_{ij} the reversal potential of the channel the presynaptic cell opens. The sign of a synapse lives entirely in the driving force $E_{ij} - v_i$, never in W_{ij} .

The model is a graph network with the same connectivity:

$$\frac{dv_i}{dt} = f_\theta(v_i, a_i, \widehat{\text{msg}}_i, I_i), \quad \widehat{\text{msg}}_i = \sum_{j \in \mathcal{N}(i)} \widehat{W}_{ij} g_\phi(v_j, a_j, v_i, a_i), \quad (2)$$

where a_i is a learned per-neuron embedding and $\widehat{W}_{ij}$ a learned per-edge weight. Only dv_i/dt is supervised. *Nothing in the loss observes $\widehat{\text{msg}}$* , and that is the whole difficulty: the message is an internal variable, and any transformation of it that f_θ can undo leaves the training objective exactly unchanged.

1.1 An affine gauge

We suppose that the model's message is related to the truth by

$$\widehat{\text{msg}}_i = \frac{1}{k_i} \text{msg}_i + \beta_i. \quad (3)$$

Write the model's update in the same form as the generator's, which is what the update fit of §2.2 measures:

$$\frac{dv_i}{dt} = T_i \left[(V_i - v_i) + G_i \widehat{\text{msg}}_i + f(I_i) \right]. \quad (4)$$

Substituting (3) into (4) and expanding puts the model's update in terms of the true message,

$$\frac{dv_i}{dt} = -T_i v_i + \frac{T_i G_i}{k_i} \text{msg}_i + T_i (V_i + G_i \beta_i) + T_i f(I_i), \quad (5)$$

while the generator (1), divided by τ_i and grouped the same way, reads

$$\frac{dv_i}{dt} = -\frac{1}{\tau_i} v_i + \frac{1}{\tau_i} \text{msg}_i + \frac{V_i^{\text{rest}}}{\tau_i} + \frac{I_i}{\tau_i}. \quad (6)$$

The two hold for every trajectory the network produces, so the coefficients of v_i , of msg_i and the constant must agree one by one. Matching them gives three identities:

$$\text{message:} \quad T_i G_i \frac{1}{k_i} = \frac{1}{\tau_i} \quad \implies \quad k_i = \tau_i T_i G_i, \quad (7)$$

$$\text{leak:} \quad T_i = \frac{1}{\tau_i} \quad \implies \quad \tau_i = 1/T_i, \quad (8)$$

$$\text{constant:} \quad T_i V_i + T_i G_i \beta_i = \frac{V_i^{\text{rest}}}{\tau_i} \quad \implies \quad V_i^{\text{rest}} = \tau_i T_i V_i + k_i \beta_i. \quad (9)$$

2 The readout, as least squares

With the identities in hand the question becomes an empirical one: does the trained network obey the generator's equation (1) at all, and if so with which constants? The two are answered by the same fit. Impose the generator's form on what the network computes, ask least squares for the constants that fit it best, and read the residual: the coefficients are the recovered parameters, and the R^2 is how well the hypothesis holds on that edge or that neuron. An edge whose message cannot be written as $W_{ij} \text{act}(v_j) (E_{ij} - v_i)$ reports a low R^2 and its constants mean nothing; one that fits at 0.999 has a message the generator could have produced, and then the only question left is whether the constants are the right ones.

2.1 Per edge: W_{ij} and E_{ij}

Write $u_t = \text{act}(v_j(t))$. The generator's synaptic form, multiplied out, is

$$\text{msg}_{ij}(t) = W_{ij} u_t (E_{ij} - v_i(t)) + C_{ij} = \underbrace{(W_{ij} E_{ij})}_{b_1} u_t + \underbrace{(-W_{ij})}_{b_2} (u_t v_i(t)) + \underbrace{C_{ij}}_{b_3}, \quad (10)$$

linear in (b_1, b_2, b_3) . The target is the model's OWN per-edge message, $\widehat{\text{msg}}_{ij}(t)$, taken from its forward pass, so the fit asks what constants the generator would need to produce what the network computes. Stack the n usable frames $t_1 \dots t_n$ of this edge into a design matrix A and the model's messages into y :

$$A = \begin{pmatrix} u_{t_1} & u_{t_1} v_i(t_1) & 1 \\ u_{t_2} & u_{t_2} v_i(t_2) & 1 \\ \vdots & \vdots & \vdots \\ u_{t_n} & u_{t_n} v_i(t_n) & 1 \end{pmatrix} \in \mathbb{R}^{n \times 3}, \quad b = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix}, \quad y = \begin{pmatrix} \widehat{\text{msg}}_{ij}(t_1) \\ \vdots \\ \widehat{\text{msg}}_{ij}(t_n) \end{pmatrix}. \quad (11)$$

The problem is $\min_b \|Ab - y\|_2^2$, whose normal equations $A^\top A b = A^\top y$ give

$$b = (A^\top A)^{-1} A^\top y, \quad W_{ij} = -b_2, \quad E_{ij} = -b_1/b_2, \quad C_{ij} = b_3. \quad (12)$$

$A^\top A$ is 3×3 whatever n is, and its six distinct entries are sums over frames — $\sum u^2$, $\sum u^2 v_i$, $\sum u$, and so on — so the frames stream and nothing of size $n_{\text{edges}} \times n$ is ever held. All 434,112 edges are then solved together as a batch of 3×3 systems, in seconds.

Which rows are usable. Three unknowns, so eight rows are ample. A row is *usable* when the presynaptic cell is above the activity floor, $u_t = \text{act}(v_j(t)) > 0$; below it the row is $[0, 0, 1]$ and constrains only the offset. Frames are drawn where the presynaptic cells are actually on, and a second sample tops up the edges still short, so 99.6% of the 434,112 edges are fitted.

Why E_{ij} is the fragile one. W_{ij} is a coefficient; E_{ij} is a ratio of two, so its relative error is $(\sigma_E/E)^2 \approx (\sigma_{b_1}/b_1)^2 + (\sigma_{b_2}/b_2)^2$ and diverges as $b_2 \rightarrow 0$. And b_2 is small not when the synapse is weak but when the postsynaptic voltage barely moves: if $v_i(t) \approx c$ over the usable frames then $u_t v_i(t) \approx c u_t$, the two columns are nearly the same vector, and only the combination $b_1 + c b_2$ is determined. The guard is a t -test on that coefficient, from the same normal equations:

$$\text{se}(b_2) = \sqrt{\hat{\sigma}^2 [(A^\top A)^{-1}]_{22}}, \quad t = \frac{|b_2|}{\text{se}(b_2)}, \quad E_{ij} \text{ kept only where } t \geq 3. \quad (13)$$

Everything below it is recorded as unidentified rather than dropped silently. On the conductance run used throughout: 432,517 of the 434,112 edges are fitted (99.6%), and of those, 405,503 clear the gate and report a reversal (93.4% of all edges). The two counts differ by arm — an earlier model identified only 29.4% — so two runs' E scores are comparable only when their identified counts are.

2.2 Per neuron: T_i , V_i , G_i

The same least-squares approach on the update. Taking $f(I)$ linear, (4) expands to a four-column regression of the model's own dv_i/dt on its own inputs,

$$\widehat{\frac{dv_i}{dt}} = a_0 + a_1 v_i + a_2 \widehat{\text{msg}_i} + a_3 I_i, \quad T_i = -a_1, \quad V_i = a_0/T_i, \quad G_i = a_2/T_i, \quad (14)$$

again $(A^\top A)^{-1} A^\top y$, now 4×4 per neuron and accumulated over frames so nothing of size $N \times n_{\text{frames}}$ is ever held. The fit's R^2 is reported: it is how far the model's update is from affine in the message, and a low value invalidates T , V and G together.

2.3 From the fitted coefficients to the circuit

The two fits return numbers about the model: per edge (b_1, b_2, b_3) , per neuron (a_0, a_1, a_2, a_3) , and from them T_i , V_i , G_i , W_{ij}^{fit} , E_{ij} , C_{ij} . The identities (7)–(9) are what turn those into the generator's parameters, and they are used in this order:

$$(8) \quad \tau_i = 1/T_i = -1/a_1 \quad \text{the time constant, directly,} \quad (15)$$

$$(7) \quad k_i = \tau_i T_i G_i = G_i = a_2/T_i \quad \text{the message scale, which } W_{ij} \text{ is in units of,} \quad (16)$$

$$(9) \quad V_i^{\text{rest}} = \tau_i T_i V_i + k_i \beta_i \quad \text{the resting level, offset included.} \quad (17)$$

So τ comes straight out of the update fit, W_{ij} needs the scale from (7) applied to the edge fit, and V^{rest} needs the offset that (9) says the update hid in it — the two corrections below. E_{ij} needs neither.

W_{ij} : divide by the scale. By (7) the message is in units of $1/k_i$ with $k_i = \tau_i T_i G_i$, and the fitted b_2 inherits that, so the weight in the generator's units is

$$W_{ij}^{\text{corrected}} = k_i W_{ij}^{\text{fit}} = \tau_i T_i G_i (-b_2), \quad \tau_i = 1/T_i \Rightarrow k_i = G_i, \quad (18)$$

where τ_i is the model's own, from (8). Without this the same weights score $R^2 = -2.14$ where the corrected ones score 0.995 — the uncorrected number is kept as `Wij_R2_uncorrected` so the difference is visible rather than assumed.

V^{rest} : add the offset back. By (9) the level the update fit reads is short by exactly the neuron's own incoming message offset:

$$V_i^{\text{rest, corrected}} = \underbrace{\tau_i T_i V_i}_{\text{from the update fit}} + \sum_j C_{ij} \underbrace{k_i \beta_i}_{\text{from Stage 1}}, \quad (19)$$

the per-neuron sum of the per-edge offsets C_{ij} the edge fits already produced. On fold 0 of the noise-0.05 current run this moves $V^{\text{rest}} R^2$ from 0.933 to 0.958; the uncorrected value is kept as `V_rest_R2_uncorrected`. Note the asymmetry with W_{ij} : the scale is divided out of the weights, the offset is added into the resting potential, and both are properties of the message gauge rather than errors of the parameter.

E_{ij} needs neither. A reversal is the voltage at which the message vanishes — a root — and a root does not move when the function around it is rescaled, or shifted by a constant the fit accounts for separately as b_3 . It comes out of (12) in absolute units already.

msg_i follows the same scale. The aggregated message is reported as $k_i \widehat{\text{msg}}_i$, the model’s own aggregate through the gauge the update fit measured — one multiplication, no derivative of f_θ anywhere.

3 A worked example

Figure 1 shows the readout applied to neuron 2895 of a conductance network trained at process noise $\sigma = 0.05$ on the flyvis connectome ($N = 13,741$ neurons, 434,112 edges).

The V^{rest} identity closes. Equation (9) predicts $V^{\text{rest}} = \tau TV - b$ where $b = -k\beta$ is the fitted offset of panel (c). Substituting: $-0.083 + 2.334 = +2.251$ against the generator’s $+2.910$ — 77% of the resting potential recovered, of which essentially all comes from the message’s offset and almost none from the update’s own level $V = -0.21$. Reading V alone as V^{rest} , as a naive fit of (4) invites, would be wrong by 3 V.

The per-edge offsets sum to the per-neuron one. Panel (c) reports $+2.334$ measured on the total message against $+1.510$ from summing the eight fitted C_{ij} ; the gap is the three synapses the model pruned, which have no fit to contribute one. The two stages agree on a quantity neither of them imposes.

4 Population results

Table 1: The same trained model, read three ways. “Chain” is the previous estimator (a per-edge line through m/v_j with the g_ϕ and f_θ gain corrections). “Blind” and “diagnostic” are the two gauge conventions of §2.3. W statistics are over the 321,774 edges the fit accepted; R^2 is outlier-filtered (383 outliers, 0.12%).

	chain	template, blind	template, diagnostic
$W_{ij} R^2$	—	+0.273	+0.691
W_{ij} Pearson r	—	0.238	0.796
W_{ij} median relative error	—	25.2%	7.3%
W_{ij} slope	—	0.582	0.760
E_{ij} median per-edge fit R^2	0.812	0.9997	0.9997
E_{ij} RMSE (V)	—	7.76	7.76
τR^2	0.912	0.925	0.925
$V^{\text{rest}} R^2$	0.787	0.792	0.792
clustering accuracy	0.490	0.583	0.662
clustering NMI	0.714	0.828	0.838

Three readings of Table 1 deserve emphasis.

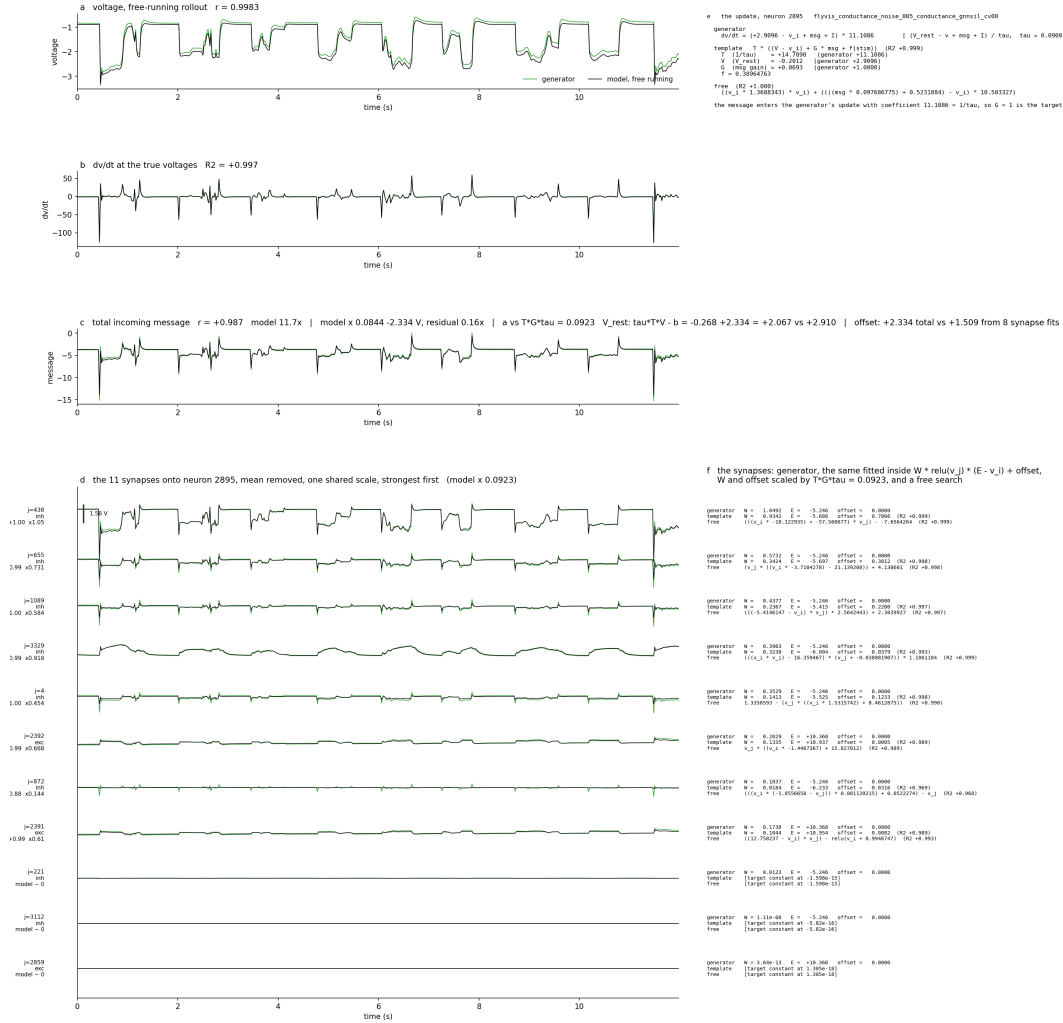


Figure 1: One neuron, term by term. **(a)** free-running rollout, generator in green and model in black, $r = 0.998$. **(b)** dv/dt at the true voltages, $R^2 = 0.997$. **(c)** the total incoming message: the learned one after the affine correction of (3), against the generator's. The heading reports the fitted scale 0.0844 against $TG\tau = 0.0923$ from panel (e) — two independent routes to the same number, agreeing to 9% — and the offset $b = -2.334$ V. **(d)** each of the 11 synapses separately, mean removed, on one shared voltage scale so that a synapse the model dropped looks flat rather than being magnified to full height. **(e)** the update fit of (??): $T = 14.79$ against the generator's $1/\tau = 11.11$, $G = 0.0693$ against a target of 1. **(f)** per synapse, the generator's ($W, E, \text{offset} = 0$) against the template fit of (??), and a free symbolic search for comparison.

The form is identified; the constant inside it is not. The median per-edge R^2 of the conductance template is 0.9997: the model’s message *is* of the form $W \text{act}(v_j)(E - v_i) + C$ to four decimal places. The reversal implied by that form nonetheless has an RMSE of 7.76 V across the 66 distinct values the generator uses. Reporting the fit quality as if it were recovery would be reporting the extraction’s goodness of fit; both numbers belong in any claim about reversal potentials.

What separates blind from diagnostic is τ , and only τ . The two columns differ by nothing but which time constant closes (??). Since τ is recovered at $R^2 = 0.925$, one might expect the effect to be small; it is not, because k_i multiplies an entire row of the connectome, so each neuron’s τ error scales all of its synapses coherently. The median τ is 37% too large, and the W statistics inherit exactly that. *This makes a prediction that can falsify the whole framing:* a regulariser that pins $G_i \rightarrow 1$ forces $k_i = \tau_i T_i G_i \rightarrow 1$ and removes τ from the conductance readout altogether, so the two columns should converge. If they do not, the gauge is not the whole story.

The population number is dominated by synapses that do not exist. Figure 2 shows why the per-neuron picture (median learned/true 1.39, interquartile range 1.22) and the population R^2 (0.273) tell different stories. Half the fitted edges have a true conductance below 10^{-8} (the first five deciles run from 2×10^{-15} to 8.5×10^{-9}) — the generator has no synapse there — and the model gives them conductances up to 10^{-2} . They dominate the sum of squares while carrying no biology. A second, smaller population is whole postsynaptic neurons whose rows collapsed, visible as vertical streaks at ratios of 10^{-6} and below.

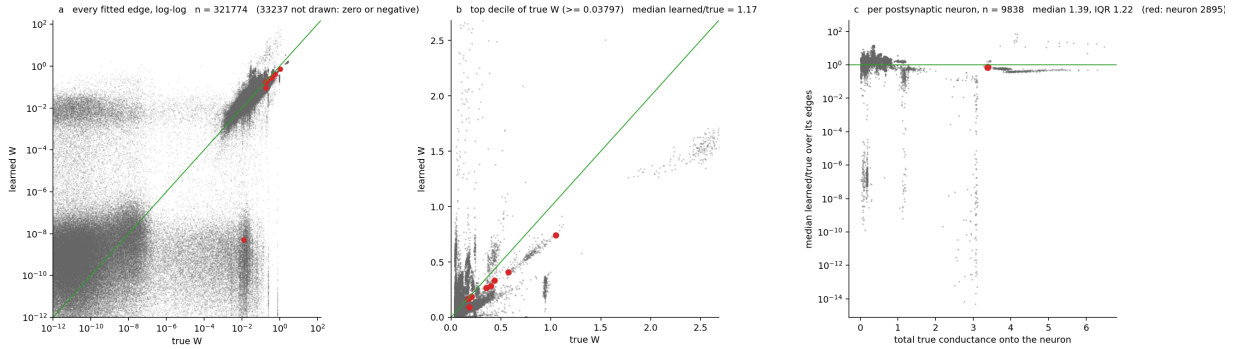


Figure 2: Learned against true conductance, blind gauge. **(a)** every fitted edge, logarithmic axes, with neuron 2895’s synapses in red; the diagonal ridge is the recovered population and the cloud at lower left is the phantom one. **(b)** the top decile of true conductance on linear axes, median learned/true 1.17. **(c)** per postsynaptic neuron, the median ratio over its edges against the total conductance it receives; the bulk sits at 1.39 and the vertical streaks are collapsed rows.

5 Is the method sound?

What it assumes. (i) That act is known. It is given from the generator, so on real data this becomes an assumption about the synaptic nonlinearity and should be stated as one; the free symbolic search in panel (f) of Figure 1 exists to check it, and a separate PySR template search — which fits act rather than assuming it — is run on a handful of neurons for exactly that purpose. (ii) That f_θ is approximately affine in the message over the observed range; measured, not assumed ($R^2 = 0.9994$, ratio to autograd 0.9994). (iii) That the generator gives each neuron two reversals, used only by the pooling step, which can be disabled.

What it does not do. It does not make an unidentifiable quantity identifiable. The offset β_i is measured here against the generator’s message, which is an *account* of the error, not a blind recovery; only a prior that pins the message’s level — a penalty on g_ϕ at silent presynaptic input — makes it identifiable during training. The same holds for the scale: the readout undoes k_i after the fact, and a penalty tying $\partial f_\theta/\partial \text{msg}$ to $-\partial f_\theta/\partial v$ is what would remove it during training.

What would falsify it. The convergence prediction of §4; the agreement of $\sum_j C_{ij}$ with the independently fitted total offset, which currently holds to the pruned synapses; and the agreement of the fitted $T_i G_i$ with autograd, which would break first if the update stopped being affine in the message.

6 Implementation

The readout is `metrics.extract_template_params`, scored through the same `score_recovery` as every other estimator so that the two readouts of a run differ only in `Wij_estimator` and can be compared line by line. Per-edge fits are batched `numpy` solves of stacked 3×3 normal equations; the per-neuron update is a batched 4×4 solve accumulated over frames. Cost on the example run: all 434,112 edges and 13,741 neurons in under a minute on one L4, against the ~ 300 s per neuron a symbolic search over act would take. The figures in this note are generated by `GNN_Main.py -o plot` and `tools/wij_scatter.py`.